Architectural AI Design

Web Prototype

Table of Contents

Table of Contents 2
Architectural AI Design Web Prototype — 4-Day Delivery Plan 4
1. Executive recommendation 4
What the four-day PoC will demonstrate 4
Explicit PoC boundaries 4
2. Recommended technology choices 5
Vendor capability notes to validate on Day 1 6
3. Target architecture and end-to-end flow 7
Customer journey 8
Suggested project status model 8
4. Functional design by customer requirement 8
4.1 Empty plot → building plan 8
4.2 Existing apartment → interior concepts 9
4.3 Existing house → landscaping concepts 9
5. Pricing calculator and offline payment workflow 9
Pricing model 9
Manual payment workflow 10
6. Data model for the PoC 11
`projects` 11
`quotes` 11
`payments` 11
`generation_jobs` 11
`design_assets` 11
`pricing_rules` / `site_settings` 11
`audit_events` 11
7. Four-day timeline 12
Work to defer if the schedule slips 12
Day-by-day detail 12
Day 1 — Product shell and architecture 12
Day 2 — Voice and English brief 13
Day 3 — Image generation, calculator, payment submission 13
Day 4 — Gating, quality assurance, deployment 13
8. Phased implementation plan and coding-agent prompts 13

Phase 0 — Repository inspection, configuration, and vertical-slice plan 13
Phase 1 — Professional landing page and three input journeys 14
Phase 2 — Gnani voice path: STT, transcript review, and TTS 15
Phase 3 — Requirement normalization and approved English brief 15
Phase 4 — Plan renderer and AI concept generation 16
Phase 5 — Quote calculator, offline payment evidence, and admin operations 16
Phase 6 — Release gate, UX polish, QA, and deploy 17
9. Prompt templates used by the running application 17
9.1 Requirement-to-brief system prompt 17
9.2 Empty-plot 2D layout data prompt 18
9.3 Image-generation prompt — empty plot / exterior concept 18
9.4 Image-generation prompt — existing apartment interior 18
9.5 Image-generation prompt — existing house landscaping 18
9.6 Image-model request wrapper prompt 19
10. UI quality requirements 19
Visual direction 19
Suggested main navigation 19
Must-have states 20
11. QA and acceptance checklist 20
Functional checks 20
Security and privacy checks 20
Demo readiness 20
12. Key risks and mitigation 21
13. Day-4 demonstration script (approximately five minutes) 21
14. Handoff and next phase after the PoC 21
15. Reference links 22
Final architecture decision 22

Note: If the page numbers are not populated when the document opens, update the Table of Contents in Microsoft Word.

Architectural AI Design Web Prototype — 4-Day Delivery Plan

Document type: Solution architecture, phased implementation plan, and coding-agent prompts

Delivery target: Four working days from kickoff

1. Executive recommendation

Build a **vertical-slice proof of concept (PoC)** that takes a customer from spoken requirement to an approved English design brief, calculates a quote, accepts offline-payment evidence, and releases AI-generated design concepts only after an administrator confirms receipt of payment.

A four-day delivery is realistic only if the first release is tightly scoped and API credentials are available on Day 1. It can demonstrate the end-to-end journey and a polished customer experience. It should **not** be represented as a production architectural-design platform or as a replacement for a qualified architect, structural engineer, landscape professional, or local planning authority.

What the four-day PoC will demonstrate

1. A professional landing page and three clearly separated customer journeys:
   - **Build on an empty plot:** collect plot/building requirements; produce a labelled, illustrative 2D room layout and AI-generated 3D concept views.
   - **Redesign an existing apartment:** collect room/style/budget requirements and reference photos or a floor plan; generate interior design concept images.
   - **Landscaping for an existing house:** collect outdoor-area, sunlight, climate, maintenance, and style requirements; generate landscape concept images.
2. Voice recording in the browser, speech-to-text (STT), transcript correction, requirement clarification, and text-to-speech (TTS) read-back.
3. An English design brief and prompt generated from the customer-approved requirements.
4. AI image generation through OpenRouter, using a configurable model and uploaded reference images where supported.
5. A configurable pricing calculator and downloadable/shareable quote summary.
6. Offline UPI or netbanking instructions, customer submission of a transaction reference/proof, and manual administrator verification.
7. Server-enforced payment gating: designs cannot be released until payment has been verified and the generated output has passed the release step.
8. A small admin area for pricing rules, payment review, job status, and release control.

Explicit PoC boundaries

- **2D floor plan:** render from structured room/layout data into SVG or canvas so room names and approximate dimensions remain legible. Do not ask an image model to draw precise room labels. This is a conceptual layout, not CAD/BIM output.
- **3D view:** provide AI-generated perspective concept renders in this version. These are static images, not a navigable, dimensionally accurate 3D model. Interactive geometry, IFC/Revit/SketchUp export, CAD editing, and code-compliant construction drawings are out of scope.

- **Voice coverage:** design the language selector so it can grow, but test a small agreed set of priority languages and dialects within four days. Vendor language claims do not guarantee accuracy for every local dialect, mixed-language utterance, accent, or architectural term.
- **Payment:** this is an offline/manual process, not a payment gateway. The customer submits a UTR/reference and optional receipt screenshot; an administrator verifies the credit in the actual bank/UPI account. The app must never request a bank password, card PIN, or UPI PIN.
- **No production promise:** security review, legal/tax review, comprehensive dialect testing, detailed architectural validation, high-volume load testing, and full operational monitoring are follow-on work.

2. Recommended technology choices

Layer	PoC recommendation	Reason / implementation note
Web UI	Next.js App Router, TypeScript	One web codebase for customer pages, admin pages, and server-side API routes.
UI styling	Tailwind CSS, shadcn/ui, Lucide icons	Consistent design system and fast implementation of polished forms, cards, dialogs, badges, and admin tables.
Database	Supabase Postgres	Stores projects, briefs, quotes, payment status, pricing rules, generation jobs, and release state.
Authentication	Supabase Auth	Email magic-link/OTP sign-in; admin role assigned server-side. Avoid a shared hard-coded admin password.
File storage	Supabase Storage with private buckets	Stores uploaded reference images, payment proof, generated concepts, and audio only when retention is required. Use signed URLs for access.
STT	Gnani Prisma through a server-side adapter	Current Gnani material advertises multilingual Indian speech recognition and code-switching. Confirm the exact API endpoint, languages, limits, and credentials with the account being used.
TTS	Gnani Timbre through a server-side adapter	Use language-specific TTS where available; return a text fallback if audio generation fails or the requested language is not enabled.
Brief/JSON generation	A configurable text model through OpenRouter	Convert corrected transcript and form fields into a validated, structured English brief. Keep the model slug in configuration.
Image generation	OpenRouter Image API; start with a tested image model such as `bytedance-seed/seedream-4.5`	Supports a unified image-generation interface; confirm model/provider capabilities and price at runtime. Keep a fallback model configurable.
Validation	Zod schemas + server-side business rules	Prevent malformed model output from flowing into pricing or prompt generation.
Tests	Unit tests for quote/status rules; Playwright smoke flow if time permits	Prioritize the critical payment-release gate and the three customer journeys.
Hosting	Vercel for the Next.js app + Supabase project	Suitable for a fast hosted demo. Use the hosting platform already approved by the team if different.

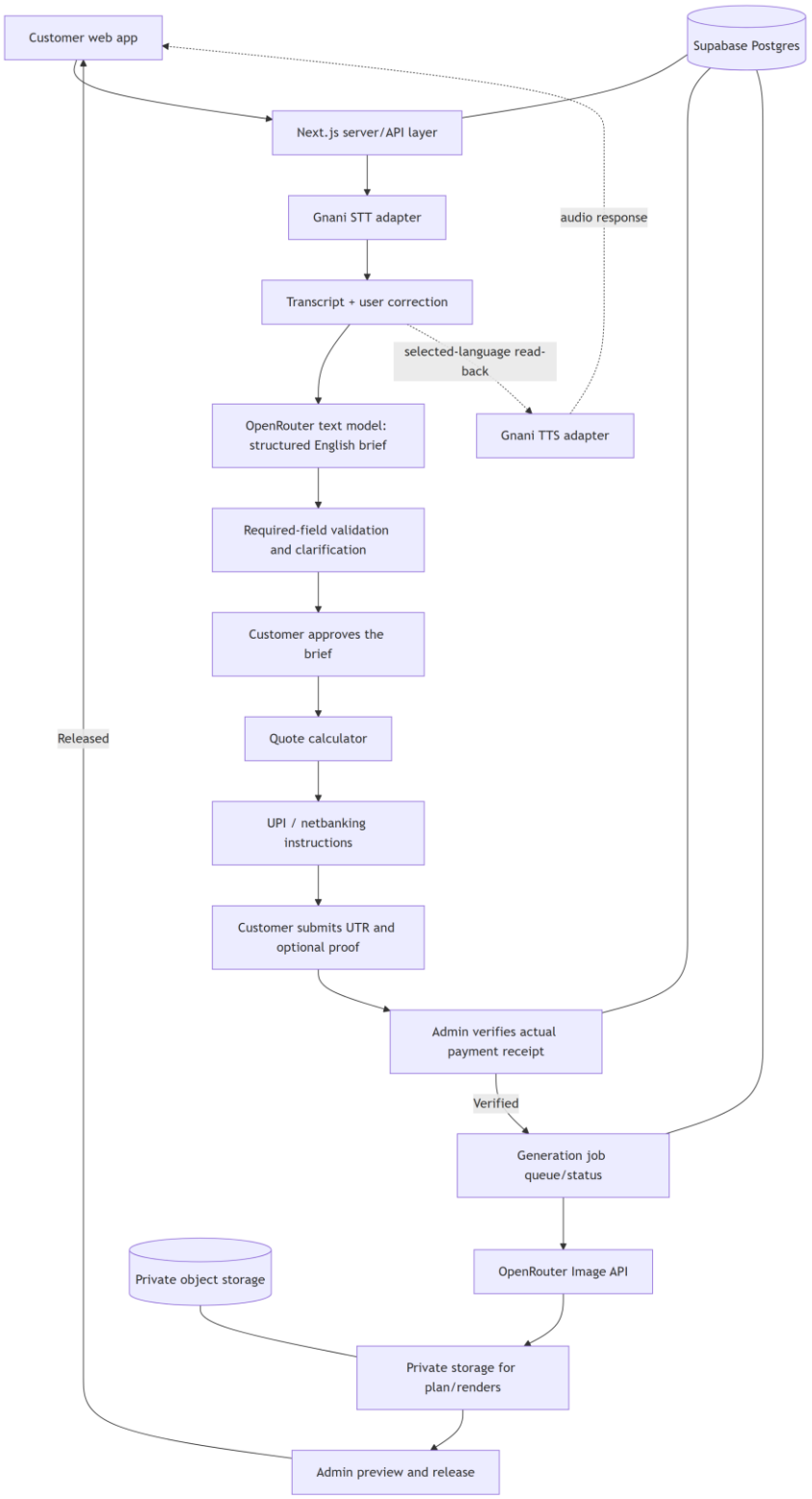
Important integration rule: API keys for Gnani, OpenRouter, database admin operations, and storage signing must remain on the server. Never expose them in browser JavaScript or `NEXT_PUBLIC_*` variables.

Vendor capability notes to validate on Day 1

- Gnani's current STT information describes Indian languages, regional accents, and code-switched speech. These are vendor-reported capabilities; test real recordings from the intended customers before claiming broad dialect coverage. See Gnani Speech-to-Text API and Gnani models.
- Gnani's current TTS material lists ten self-serve languages—Hindi, Indian English, Bengali, Tamil, Telugu, Marathi, Gujarati, Kannada, Malayalam, and Punjabi—while other model/product pages may describe a broader model capability. Check the actual account entitlement and voice list. See Gnani Text-to-Speech API.
- OpenRouter documents a dedicated image endpoint (`POST /api/v1/images`) and model discovery (`GET /api/v1/images/models`). Model parameters, reference-image support, provider availability, and prices vary; check the chosen endpoint rather than assuming every model supports the same options. See OpenRouter Image Generation and image model discovery.

3. Target architecture and end-to-end flow

Figure 1 — 3. Target architecture and end-to-end flow



Customer journey

1. Customer selects one of the three project types.
2. Customer chooses a language and records their requirements, or enters them using text. Offer both paths so the demo is not blocked by a microphone or speech-service issue.
3. The app transcribes the recording and displays the transcript for correction. Preserve the original transcript and the corrected version separately.
4. A text model converts the corrected transcript and form fields into structured requirement data and an English brief. Missing critical information appears as specific clarification questions, not invented assumptions.
5. The customer reviews and approves the brief; the app calculates a quote from server-owned pricing rules.
6. Customer follows the configured UPI/bank instructions and enters the transaction reference. Optional payment proof is uploaded to private storage.
7. An admin checks the actual account and marks the payment as verified or rejected with a reason.
8. Only verified jobs can enter generation. The app creates the 2D SVG layout, sends image prompts to OpenRouter, and saves the outputs.
9. Admin previews the outputs and clicks **Release**. The customer can then view the brief, quote, and design deliverables.

Suggested project status model

Use a server-controlled state machine rather than treating the UI as the source of truth:

`DRAFT` → `TRANSCRIPT_READY` → `BRIEF_REVIEW` → `BRIEF_APPROVED` → `QUOTED` → `AWAITING_PAYMENT` → `PAYMENT_PROOF_SUBMITTED` → `PAYMENT_VERIFIED` → `GENERATING` → `READY_FOR_REVIEW` → `RELEASED`

Additional terminal/recovery states: `PAYMENT_REJECTED`, `GENERATION_FAILED`, and `CANCELLED`.

Non-negotiable rule: a server-side transition to `GENERATING` must verify `payment_status = VERIFIED`; a server-side transition to `RELEASED` must verify both verified payment and available deliverables. Hiding a button in the UI is not sufficient security.

4. Functional design by customer requirement

4.1 Empty plot → building plan

Inputs to collect

- Plot width and length, unit, and approximate site orientation (optional).
- Desired built-up area, number of floors, and preferred room list.
- Bedroom/bathroom counts, kitchen/dining/living needs, stair requirements, parking, balcony/utility, and optional study/puja/storage room.
- Preferred style, budget band, location/climate, and optional Vastu preferences.
- Number of concept views requested.

Outputs for the PoC

- English design brief, assumptions, and unresolved questions.
- A labelled schematic 2D SVG room plan from structured room/layout data. Where dimensions are approximated, label them as indicative.
- One or more generated 3D perspective concept images for the exterior and selected interior spaces.
- Prompt and model metadata stored with the job for traceability.

Important: do not claim the rendered plan satisfies local building regulations, setbacks, fire access, structural engineering, or Vastu unless those are separately validated by a professional.

4.2 Existing apartment → interior concepts

Inputs to collect

- Existing apartment photos and/or an existing floor plan.
- Which rooms to redesign and approximate room dimensions if known.
- Style (for example, contemporary, Scandinavian, traditional Indian, minimalist), colour/material preferences, storage needs, and budget band.
- Fixed elements that must remain unchanged: windows, doors, columns, plumbing points, electrical points, and furniture to retain.
- Number of concepts/views requested.

Outputs for the PoC

- English brief per selected room.
- AI-generated concept board/renders based on the supplied reference photo(s) when the chosen model/provider supports reference inputs.
- A text summary of proposed palette, finishes, furniture, lighting, and storage ideas.

Important: generated images may alter dimensions, openings, product details, or room geometry. Show a visible concept-only disclaimer.

4.3 Existing house → landscaping concepts

Inputs to collect

- Front yard/backyard/side yard/terrace selection and approximate area.
- City or climate zone, sunlight (full/partial/shade), water availability, maintenance tolerance, and pet/child considerations if relevant.
- Existing trees/features to retain, hardscape/pathway requirements, seating, privacy, vegetable/herb garden, and style.
- Number of views requested.

Outputs for the PoC

- English landscape brief and planting/design intent.
- AI-generated landscape concept images based on the reference photo where supported.
- Suggested design elements and a note that specific plant selection should be checked for local climate, toxicity, water needs, and maintenance.

5. Pricing calculator and offline payment workflow

Pricing model

All rates should be configured in the database or admin settings—never embedded in UI components. The quote should explain each line item.

A basic model is:

```
Subtotal = Base fee + Area component + Room/zone component + Extra-view component + Optional add-ons
Total = Subtotal + Configured taxes/fees - Approved discount
```

Possible variables by project type:

Project type	Calculator inputs	Potential line items
Empty plot / building concept	Plot area, proposed built-up area, number of floors, room count, number of views	Base planning fee, floor/room complexity, extra 3D views, additional revision pack
Apartment interior	Area or number of rooms, selected rooms, style/detail tier, number of views, reference-image count	Base fee, per-room/area component, extra renders, premium detail tier, extra revisions
Landscaping	Landscape area, zone count, number of views, complexity/maintenance tier	Base fee, area/zone component, extra renders, planting-plan add-on

For this PoC, store `rate`, `unit`, `minimum_charge`, `included_quantity`, `add_on_rate`, currency, tax configuration, and effective date as configurable rule fields. Seed the environment with **clearly labelled demo-only placeholder rates** or enter approved commercial rates before the demo. Do not represent placeholder values as a real commercial price list. Freeze the computed quote breakdown when the customer accepts it so later rate changes do not silently alter an existing quote.

Manual payment workflow

1. Quote acceptance shows the business's approved UPI ID/QR and/or bank transfer details.
2. Customer transfers outside the application and submits transaction reference/UTR, transfer date, amount, and optional receipt screenshot.
3. Application creates a `PAYMENT_PROOF_SUBMITTED` record. The payment remains unverified.
4. Admin compares the details with the actual UPI/bank account and records `verified_by`, `verified_at`, confirmed amount, and an audit note.
5. If the amount/reference is incorrect, admin rejects the submission with a reason and the customer may resubmit.
6. Verified payment enables generation; it does **not** automatically make private design files public.
7. Once generated, admin inspects the output and selects **Release**. Only then may the customer access final output URLs.

Security and finance controls

- Keep payment proofs in a private bucket and serve them only through authenticated, time-limited signed URLs.
- Do not ask the customer for UPI PIN, netbanking password, OTP, or full banking credentials.
- Keep payment and release actions in an audit log.
- Show payment instructions sourced from server-side configuration; do not accept a customer-entered UPI ID as the official destination.
- Make refund/cancellation language and taxes configurable and have the business owner/accountant review it before real payments.
- This workflow is manual evidence submission, not proof that a transfer happened. Only an operator with access to the receiving account can confirm receipt.

6. Data model for the PoC

A minimal schema is sufficient. Use UUID IDs, timestamps, foreign keys, and server-side authorization checks.

`projects`

- `id`, `customer_id`, `project_type`
- `status`, `source_language`, `created_at`, `updated_at`
- `transcript_original`, `transcript_corrected`
- `requirements_json`, `english_brief_json`, `english_brief_text`
- `approved_at`, `approved_by_customer_at`

`quotes`

- `id`, `project_id`, `currency`, `line_items_json`
- `subtotal`, `tax_amount`, `discount_amount`, `total_amount`
- `pricing_rule_version`, `valid_until`, `accepted_at`

`payments`

- `id`, `project_id`, `quote_id`, `amount_claimed`, `currency`
- `method` (`UPI` or `NETBANKING`), `utr_reference`, `transfer_date`
- `proof_storage_path`, `status`, `rejection_reason`
- `verified_by`, `verified_at`, `audit_note`

`generation_jobs`

- `id`, `project_id`, `status`, `provider`, `text_model`, `image_model`
- `prompt_version`, `prompt_json`, `attempt_count`, `error_code`, `error_summary`
- `created_at`, `started_at`, `completed_at`

`design_assets`

- `id`, `project_id`, `generation_job_id`, `asset_type`
- `storage_path`, `mime_type`, `width`, `height`, `is_released`, `created_at`

`pricing_rules` / `site_settings`

- Configurable project-type rates, included quantities, add-ons, taxes, payment instructions, supported languages, model IDs, and policy copy.

`audit_events`

- Actor, action, project/payment ID, timestamp, and minimal metadata for payment verification, rejection, generation, and release actions.

Do not store bank login information. Avoid retaining raw audio longer than necessary; make retention policy explicit. If audio is not needed after transcription, delete it after a successful transcript unless the customer has clearly consented to retention.

7. Four-day timeline

Assumption: Day 1 begins Monday, 12 October 2026, and work continues for four consecutive working days through Thursday, 15 October 2026. Adjust the calendar dates if the kickoff differs. The schedule assumes one capable full-stack developer with some product/UI support; two developers can parallelize UI and integration work after the schema contract is agreed.

Day	Hours (approx.)	Focus	Concrete deliverables / exit check
Day 1 — Foundation and UI	8	Confirm scope/credentials; create app; implement visual system, landing page, and the three project journeys	Deployed or locally running shell, responsive landing page, project-type selection, complete typed forms, Supabase tables/storage configured, mock STT/image adapters available for fallback.
Day 2 — Voice and requirement brief	8	Browser recording, Gnani STT/TTS adapters, transcript correction, structured brief generation, missing-field validation	At least one live voice path works for each agreed target language; transcript can be edited; approved form/transcript becomes validated English brief; TTS read-back works or gracefully falls back to text.
Day 3 — Design generation, quote, payment submission	8	OpenRouter image adapter, SVG floor-plan renderer, gallery, pricing rules/quote; payment instructions and proof submission; basic admin queue	All three flows create demo deliverables using live models or labelled mock assets; calculator shows itemized breakdown; payment submission creates a pending record; outputs remain private.
Day 4 — Verification gate, QA, deploy, demo	8	Admin payment verification/rejection, server-side release gating, error states, regression tests, production-like deployment and scripted demo	Unverified project cannot generate/release through direct API calls; verified project can generate; admin release unlocks customer assets; three happy paths and failure paths smoke-tested; setup/runbook and demo data ready.

Work to defer if the schedule slips

1. Interactive 3D model manipulation and export to CAD/BIM.
2. Automated payment gateway/webhook integration.
3. Full regional-language and dialect certification; test priority locales first.
4. Multiple image-model orchestration, bulk generation, queue infrastructure, and detailed cost optimisation.
5. Advanced user accounts, team/architect collaboration, revision history, and production analytics.

Day-by-day detail

Day 1 — Product shell and architecture

- Confirm the project types, required fields, two priority voice locales, approved payment instructions, and demo pricing rules.
- Configure OpenRouter and Gnani credentials; perform isolated STT/TTS/image smoke tests before wiring the UI.
- Build the responsive landing page and three-card project selector.
- Build each request form with conditional fields, image uploads, sample briefs, budget/style controls, and a clear progress indicator.
- Create the database schema and private storage buckets.
- Add local/mock adapter mode so development and the final demo can continue if an external API is temporarily unavailable. Mock results must be visibly marked as demo data.

Day 2 — Voice and English brief

- Capture microphone audio using browser `MediaRecorder`; request browser permissions with a clear explanation.
- Send audio from the browser to a server route; the route calls Gnani using server-side credentials.
- Display the returned transcript and let the customer correct names, measurements, room labels, and architectural terminology.
- Merge corrected transcript with typed form fields; call a text model through OpenRouter and validate the JSON response with Zod.
- Ask only targeted follow-up questions for required missing details. Never silently invent dimensions or a room count.
- Create a selected-language read-back using TTS where supported; otherwise provide the same text on screen.
- Add a visible customer approval step before calculating a final quote.

Day 3 — Image generation, calculator, payment submission

- Add OpenRouter Image API wrapper, configured image model ID, request timeout, retry policy, cost logging, and clear generation-failed state.
- For empty plots, create a labelled SVG plan from structured room data. Treat the image-model render as a concept image, not the source of room labels/dimensions.
- For apartment and landscaping paths, forward selected reference images only after checking file type/size and provider support for reference inputs.
- Save image outputs to private storage; persist model/prompt metadata and generation state.
- Implement server-calculated quote line items from admin-configured rates. The browser may preview the quote, but it must not set the final amount.
- Present official UPI/bank details, accept the transfer reference and optional payment proof, and set payment state to pending.
- Build the first admin queue view for pending payments and generation jobs.

Day 4 — Gating, quality assurance, deployment

- Implement admin-only verify/reject actions and append immutable audit events.
- Add server-side checks to all generation, asset-access, and release routes.
- Add failure handling for missing credentials, rate limits, invalid model responses, unsupported languages, oversized uploads, and generation timeouts.
- Test the three happy paths plus direct API attempts against unpaid jobs.
- Deploy the application, configure production environment variables, and verify storage privacy and role-based access.
- Prepare a demo account, safe sample images, a sample payment marked verified by an admin, generated sample outputs, and a five-minute demonstration script.

8. Phased implementation plan and coding-agent prompts

The following prompts can be supplied one at a time to a coding assistant in the repository. Each prompt intentionally limits scope; review the diff and run the specified checks before moving on. If a repository already exists, adapt the plan to its established conventions rather than replacing its architecture without a reason.

Phase 0 — Repository inspection, configuration, and vertical-slice plan

Time box: Day 1, first 1–2 hours **Deliverables:** written repository assessment, `.env.example`, architecture decisions, working app shell, health/readiness check, and provider smoke tests.

Prompt to the coding assistant

> Act as a senior full-stack engineer implementing a four-day architectural-design AI PoC. First inspect the existing repository, package manager, scripts, application routes, styling system, and tests. Do not overwrite or restructure unrelated code. Propose the smallest workable architecture using Next.js App Router + TypeScript, Tailwind/shadcn UI, Supabase Postgres/Auth/Storage, Gnani adapters for STT/TTS, and OpenRouter adapters for text/image generation. If the repository already uses a sound equivalent, retain it and explain why.
>
> Implement only the foundation in this phase: clean environment-variable loading, an `.env.example` with placeholders (no secrets), shared typed config, a health route, structured server logs, and interfaces for `SpeechToTextProvider`, `TextToSpeechProvider`, `BriefProvider`, and `ImageGenerationProvider`. Add mock adapters that are explicitly labelled as mocks. Add a README section covering local run, required credentials, and smoke-test commands. Do not implement customer workflows yet.
>
> Acceptance criteria: app starts; health route responds; no secret is committed or exposed to the browser; lint/type-check passes; existing unrelated changes remain untouched. At the end, report files changed, commands run, unresolved provider credential/API questions, and the next smallest step.

Phase 1 — Professional landing page and three input journeys

Time box: Day 1, remaining time **Deliverables:** high-quality responsive landing page; three project cards; dedicated request forms; accessible upload and recording entry points.

Prompt to the coding assistant

> Build the customer-facing UI for an AI architectural concept studio. Use a polished professional visual language suitable for architects and homeowners: warm off-white background, charcoal typography, restrained deep-green or terracotta accent, large editorial headings, generous spacing, consistent card radii, subtle shadows, clear iconography, and responsive layouts. Avoid an overcrowded dashboard aesthetic. Use accessible contrast, keyboard focus states, useful empty/error states, and mobile-friendly forms.
>
> Landing page sections: hero with primary CTA “Start a design brief”; three service cards (New Build on Empty Plot, Interior Refresh, Landscape Design); a simple “How it works” section; concept-only disclaimer; and footer. Create a short multi-step customer journey for each service type.
>
> Empty plot fields: plot dimensions/unit, approximate built-up area, floors, rooms, beds/baths, kitchen/dining/living, stairs/parking, style, budget band, optional Vastu preference, desired views. Interior fields: areas/rooms to redesign, style, budget, colour/material preferences, fixed items to preserve, number of views, photo/floor-plan uploads. Landscaping fields: area/type, city/climate, sun exposure, water availability, maintenance level, features to retain, preferred elements/style, views, reference-photo uploads. Include text input plus a “Record requirements” entry point; wire only a disabled or clearly marked placeholder if speech integration is not ready.
>
> Keep all fields typed and validated. Do not invent price rates. Include a reusable step indicator, save-draft affordance if practical, visible image-upload limits, and clear “conceptual designs are not construction drawings” language. Use realistic demo copy, not lorem ipsum. Run lint/type-check and report any incomplete interactions.

Phase 2 — Gnani voice path: STT, transcript review, and TTS

Time box: Day 2, first half **Deliverables:** browser audio capture, secure server proxy, transcript editor, supported-language selector, TTS read-back/fallback.

Prompt to the coding assistant

> Implement voice requirements capture using the existing provider interfaces. Use browser `MediaRecorder` with explicit microphone-permission messaging and a stop/cancel/re-record control. Upload audio to a protected server route; never call Gnani directly from client-side code and never expose its API key.
>
> Implement `GnaniSpeechProvider` behind a server-only adapter. Keep API base URL, model/product, language code, and credentials in server environment/configuration. Gnani has product-specific API examples, so follow the endpoint and authentication scheme for the credentials actually supplied; do not assume the legacy gRPC endpoint and current REST endpoint use the same protocol. Add timeout handling and a friendly retry/error state. Store original transcript and customer-corrected transcript separately. Add a language selector driven by configuration; initially validate the two languages chosen by the product owner, while allowing more vendor-enabled languages to be configured later.
>
> Add TTS read-back for the approved summary or clarification questions. Select a voice only from the configured voice list for the chosen language; if unsupported or the API fails, leave the text visible and explain that audio is unavailable. Do not claim every regional dialect is supported. Add consent text and delete raw audio after transcription unless retention was explicitly configured. Include tests for missing API key, malformed response, unsupported language, and API timeout.

Phase 3 — Requirement normalization and approved English brief

Time box: Day 2, second half **Deliverables:** structured JSON for all three project types, clarification questions, English brief, validation, customer approval.

Prompt to the coding assistant

> Implement a server-side `BriefProvider` through OpenRouter. Read the corrected transcript and typed project form; produce a validated JSON object specific to `EMPTY_PLOT`, `INTERIOR`, or `LANDSCAPE`. Define Zod schemas and reject malformed model output. Never allow model-generated prices or payment status.
>
> Required output shape: `projectType`, `sourceLanguage`, `requirements`, `dimensionsAndUnits`, `roomOrZoneList`, `stylePreferences`, `budgetBand`, `referenceAssetIds`, `missingRequiredFields`, `assumptions`, `englishBrief`, `customerReadbackText`, and `imagePrompts`. Missing values should be `null` or listed in `missingRequiredFields`, not fabricated. For empty plots, include rooms with approximate area only if supplied or explicitly labelled as an assumption. For apartment and landscape projects, keep existing fixed features/objects that the customer marks as must-preserve.
>
> Add a review screen that presents the transcript, structured summary, assumptions, missing details, and editable English brief. Generation and quote acceptance must be disabled until required fields are resolved and the customer approves the brief. Keep the prompt template version and model ID in the project metadata. Add schema tests and at least three fixture tests, one for each project type.

Phase 4 — Plan renderer and AI concept generation

Time box: Day 3, first half **Deliverables:** labelled SVG floor plan; OpenRouter image adapter; reference-image support; result gallery and generation states.

Prompt to the coding assistant

> Implement a provider-agnostic image generation adapter using the OpenRouter dedicated Image API. Keep `OPENROUTER_API_KEY` and model IDs server-side. Discover/verify the selected model's endpoint capabilities for image size/aspect ratio/reference-image input; make the image model configurable and retain a fallback. Log model, prompt version, returned usage/cost if provided, duration, and job status. Do not log secrets or raw private image bytes.
>
> For empty plots, implement a deterministic SVG schematic generated from validated room data. Draw room rectangles/partitions, room labels, and approximate dimension labels from the structured requirements. Include a prominent “Illustrative layout only—not CAD or a construction drawing” watermark/legend. If the room arrangement cannot be represented safely, show a simple schematic and a limitations note rather than asserting code compliance.
>
> For interior and landscape projects, send selected customer images as reference inputs only when supported by the chosen provider. Save generated results to a private storage bucket and present them in a gallery with project type, “AI concept” label, prompt/model metadata hidden from customers by default, retry action, and a clear failure state. The API must reject requests unless the project has a verified payment state. Do not expose generated asset URLs before release. Include tests for provider error, empty image result, unsupported reference input, and unpaid project denial.

Phase 5 — Quote calculator, offline payment evidence, and admin operations

Time box: Day 3, second half into Day 4 morning **Deliverables:** configurable rates, itemized quote, payment-proof submission, admin verification/rejection, audit trail.

Prompt to the coding assistant

> Implement pricing and manual payment operations with server-side authority. Store project-type pricing rules and official UPI/bank details in protected configuration/database records. The quote engine should produce an itemized breakdown from project type, area/room/zone count, included quantities, selected render count, add-ons, configured minimum charge, discount, and tax settings. Use decimal-safe currency arithmetic; do not calculate the final amount only in the browser. Persist a pricing-rule version and immutable accepted quote snapshot.
>
> Add the customer payment page with official UPI QR/ID or bank transfer instructions, accepted quote total, fields for UTR/reference, transfer date, amount, method, and optional image proof. Upload proof to a private bucket. On submission, set the payment to `PAYMENT_PROOF_SUBMITTED`, not verified. Never collect bank login, password, OTP, card PIN, or UPI PIN.
>
> Add an admin-only payment queue to review proof using signed URLs and compare it with the actual receiving account. Admin can verify or reject with reason. Protect admin routes on the server and verify role membership on every mutation. Record who changed the status, when, and why in an audit table. A customer

must not be able to modify payment status by posting arbitrary JSON. Add tests that ensure an unverified payment cannot start generation or release an asset.

Phase 6 — Release gate, UX polish, QA, and deploy

Time box: Day 4, remaining time **Deliverables:** end-to-end status workflow, responsive polish, private asset access, smoke tests, hosted demo and handoff notes.

Prompt to the coding assistant

Finish the four-day architectural AI PoC without expanding scope. Implement a server-enforced state machine: `DRAFT`, `TRANSCRIPT_READY`, `BRIEF_REVIEW`, `BRIEF_APPROVED`, `QUOTED`, `AWAITING_PAYMENT`, `PAYMENT_PROOF_SUBMITTED`, `PAYMENT_REJECTED`, `PAYMENT_VERIFIED`, `GENERATING`, `GENERATION_FAILED`, `READY_FOR_REVIEW`, `RELEASED`, and `CANCELLED`. Reject invalid transitions on the server. Only verified payment can start generation; only a verified, successfully generated project can be released by an authorized admin.

Ensure customer asset routes return signed URLs only when the authenticated customer owns the project and status is `RELEASED`, or when a properly authorized admin is inspecting it. Confirm RLS/storage policies, upload type/size restrictions, safe error messages, and server-side model/API keys. Polish mobile layouts, loading states, validation summaries, retry actions, empty screens, and the three-project navigation. Do not spend time adding features outside the agreed scope.

Run lint, type-check, unit tests, and available end-to-end smoke tests. Execute negative tests by calling generation and asset routes directly for an unpaid project. Deploy using configured environment variables and private storage. Produce a README runbook, environment checklist, known limitations, and a five-minute demo script. Report anything not tested or still mocked; never claim a mock integration is live.

9. Prompt templates used by the running application

These are starting templates for the application itself (not just coding-agent instructions). Store them as versioned server-side templates so they can be improved without rebuilding the UI.

9.1 Requirement-to-brief system prompt

```text
You are a careful architectural-design brief assistant. Convert the customer's corrected transcript and form fields
into a structured design brief for a concept-design prototype.

Rules:
- Treat the form fields and corrected transcript as customer input, not as verified measurements.
- Never invent dimensions, room counts, site constraints, budget, or local code requirements.
- Put missing required information in missingRequiredFields and ask concise follow-up questions.
- Distinguish customer-provided facts from assumptions. Every assumption must be labelled.
- Preserve the customer's must-keep features and stated preferences.
- Translate/normalize the brief into clear professional English, even when the source language contains code-
switching.
- Keep any spoken read-back concise and in the customer's selected language where supported.
- Do not quote prices, promise construction compliance, or claim engineering validation.
- Return JSON conforming exactly to the supplied schema, with no Markdown fences or extra text.

Project type: {{projectType}}
Selected language: {{sourceLanguage}}
Corrected transcript: {{transcriptCorrected}}
Form fields: {{validatedFormJson}}
Uploaded references (metadata only): {{referenceAssetMetadata}}
Required JSON schema: {{briefSchema}}
```

9.2 Empty-plot 2D layout data prompt

```
TEXT
Create an illustrative room-layout data structure for a concept floor plan from the approved brief. Return JSON only.

Requirements:
- Use only supplied dimensions, room counts, and confirmed preferences.
- Each room must have a stable ID, display name, approximate width/length only where justified, and a list of adjacent room IDs if known.
- If exact dimensions or site constraints are missing, mark dimensions as approximate/unknown and list the missing input.
- Include the requested floors and vertical circulation requirements where supplied.
- Do not claim compliance with setbacks, structural design, fire safety, local building bylaws, or Vastu.
- The server will draw the layout as an SVG; do not return an image or rely on rendered text.

Approved English brief: {{englishBrief}}
Return schema: {{layoutSchema}}
```

9.3 Image-generation prompt — empty plot / exterior concept

```
TEXT
Create a high-quality architectural concept visualization from the following approved brief.

PROJECT: New residential building concept on an empty plot.
STYLE: {{approvedStyle}}
SITE AND BUILDING FACTS: {{validatedDimensionsAndRequirements}}
ROOM PROGRAM: {{validatedRoomList}}
CLIMATE/LOCATION NOTES: {{locationAndClimate}}
BUDGET BAND: {{budgetBand}}

Visual direction: realistic architectural visualization, plausible circulation and proportions, coherent material palette, natural daylight, clean professional composition, landscaping that fits the stated climate, and a camera angle that clearly communicates the overall massing. Provide a separate interior/exterior view only when requested. Keep the building concept consistent with the brief; do not add extra storeys or extra wings that conflict with it.

Do not render room names, dimension strings, labels, signatures, watermarks, floor-plan annotations, or technical drawing symbols into the image. Do not state or imply that this concept is construction-ready or code-compliant. The application will present a separate labelled schematic and disclaimer.
```

9.4 Image-generation prompt — existing apartment interior

```
TEXT
Create an interior-design concept variation for the supplied existing-room reference image(s).

ROOM: {{roomType}}
APPROVED STYLE: {{style}}
COLOUR/MATERIAL PREFERENCES: {{paletteAndMaterials}}
BUDGET AND PRACTICAL CONSTRAINTS: {{budgetAndConstraints}}
MUST-PRESERVE ITEMS: {{fixedFeatures}}

Keep the existing room identity and preserve doors, windows, columns, room proportions, and other must-preserve features as closely as the image model allows. Introduce the requested furniture, lighting, storage, materials, and decor in a practical arrangement. Produce a polished, realistic concept image with coherent lighting and materials. Avoid labels or overlaid text.

This is a visual inspiration concept. Do not imply that hidden dimensions, structural conditions, electrical capacity, plumbing, or product dimensions have been verified.
```

9.5 Image-generation prompt — existing house landscaping

```
TEXT
Create a landscape-design concept variation based on the supplied existing-house/outdoor-area reference image(s).

AREA TYPE AND APPROXIMATE SIZE: {{areaTypeAndSize}}
LOCATION/CLIMATE: {{locationClimate}}
SUNLIGHT AND WATER AVAILABILITY: {{sunWater}}
MAINTENANCE LEVEL: {{maintenanceLevel}}
```

```
FEATURES TO RETAIN: {{featuresToRetain}}
REQUESTED ELEMENTS AND STYLE: {{landscapePreferences}}

Keep the main structure and retained features recognizable. Show a coherent, usable arrangement of planting zones,
pathways, seating or other requested elements, with believable scale, circulation, and lighting. Prefer plant forms
compatible with the provided climate and water constraints; if the location is unknown, do not assert that specific
species are suitable. Do not add text labels or botanical claims to the image.

This is a concept visualization. Plant species, pet/child safety, drainage, irrigation, and local suitability require
professional/local review.
```

9.6 Image-model request wrapper prompt

The application should send a single approved prompt to the configured OpenRouter image endpoint, plus reference images only when supported. Use these rules in code rather than relying only on prompt text:

- Validate the project is paid before calling the provider.
- Enforce maximum image count, resolution, file size, and timeout.
- Use a configured model slug and check provider capabilities/pricing against the current OpenRouter model catalog.
- Store the actual model/provider, prompt version, generation request ID if returned, actual usage/cost if returned, and output asset storage paths.
- Retry transient failures conservatively; avoid duplicate charges through an idempotency key/job record where possible.
- Do not expose private storage URLs until the project is released.

10. UI quality requirements

Visual direction

- **Brand feel:** modern, calm, premium, trustworthy—closer to an architectural studio than a generic AI chatbot.
- **Layout:** clear headline, concise copy, ample whitespace, strong typography, three distinct service cards, and persistent progress feedback within forms.
- **Inputs:** group fields into “Your space,” “Your preferences,” “Voice or text,” and “Review brief.” Avoid one extremely long form.
- **Voice UX:** language selector, visible recording state, elapsed time, re-record/cancel controls, transcript editor, and explicit consent before recording.
- **Uploads:** thumbnails, remove/replace action, file constraints, progress, and private-storage messaging.
- **Output gallery:** responsive image grid, clear AI Concept badge, brief summary, status badge, zoom/download only after release, and limitations disclaimer.
- **Payment:** prominent total and line-item breakdown, transfer instructions, UTR/reference form, pending/verified/rejected status, and clear explanation that receipt is checked manually.
- **Accessibility:** keyboard operation, visible focus, accessible labels, semantic headings, contrast, and no colour-only status indicators.
- **Responsive design:** make customer journeys usable at 360 px mobile width and desktop widths; do not make admin tables the only interface for mobile.

Suggested main navigation

`Home | New Build | Interior Design | Landscaping | My Projects` Admin-only navigation: `Admin | Payments | Generations | Pricing Settings`

Must-have states

- Idle, recording, uploading, transcribing, transcript ready, speech provider unavailable.
- Brief requires clarification, customer editing, brief approved.
- Quote ready, payment pending, proof submitted, rejected, verified.
- Generation queued, generating, complete awaiting review, failed/retryable, released.
- Empty project list, upload error, expired sign-in, unauthorized admin attempt.

11. QA and acceptance checklist

Functional checks

- [ ] The landing page shows all three project types and works at mobile and desktop widths.
- [ ] Each project type collects the required fields and can submit a text-only brief if voice is unavailable.
- [ ] Recording, STT, transcript correction, and TTS work for each Day-1-agreed priority language or clearly fall back to text.
- [ ] English brief preserves room counts/dimensions and marks missing values instead of guessing.
- [ ] Customer can edit and approve the brief before quote acceptance.
- [ ] Pricing is calculated server-side and the quote is itemized and frozen when accepted.
- [ ] UPI/netbanking proof creates a pending payment, never an automatically verified payment.
- [ ] Admin role is checked on the server for every verification/rejection/release action.
- [ ] Direct API request cannot start generation for an unverified payment.
- [ ] Customer cannot fetch proof images or generated assets belonging to another customer/project.
- [ ] Design cannot be fetched from customer routes until release; admin preview remains protected.
- [ ] A failed provider request leaves a recoverable job state and a user-friendly message.
- [ ] Mock/demo output is always labelled; live results are distinguishable from mocks.

Security and privacy checks

- [ ] No provider key appears in client bundles, logs, screenshots, or repository history.
- [ ] Private storage buckets and row-level access controls are enabled.
- [ ] Uploads have file-type and size limits; reject unexpected file types.
- [ ] Payment proof and original recordings are not publicly addressable.
- [ ] Payment verification/release actions are audit-logged.
- [ ] Basic rate limiting or request throttling protects costly model endpoints.
- [ ] Voice recording has a consent message and a documented retention/deletion policy.

Demo readiness

- [ ] One end-to-end example exists for each of the three project types.
- [ ] A successful test payment is verified by an admin using an actual test/authorized transfer or a clearly labelled demo fixture.
- [ ] At least one unpaid-project negative test is recorded.
- [ ] API credentials, environment variables, hosting, and storage access are confirmed before the live presentation.
- [ ] Fallback screenshots/mock outputs are available in case an external model provider is unavailable; never present fallback results as live model output.

12. Key risks and mitigation

Risk	Impact	Mitigation within four days
Gnani endpoint/credentials or quota unavailable	Voice workflow blocks	Confirm credentials and run a direct smoke test on Day 1; keep a visible text-entry fallback and mock mode for development only.
Dialect/accent accuracy varies	Brief has wrong room or dimension details	Show editable transcript, ask confirmation for numbers and room counts, test priority locales, and avoid claiming universal dialect coverage.
Image model changes room geometry or references	Misleading concept render	Treat renders as inspiration; produce labels through SVG; ask the customer to approve the brief; retain visible limitations.
Unstructured/unreliable model output	Invalid plan or broken quote	Force JSON schema output where available, validate with Zod, use sensible error states, and never use generated prices.
OpenRouter price/availability differs by model/provider	Unexpected cost or failed generation	Discover available models and prices through the API, cap images/resolution, log returned usage/cost, and configure a fallback model.
Payment proof is falsified or mistaken	Unpaid design released	Payment stays pending until a human confirms the transaction in the actual receiving account; enforce the state machine server-side.
Four-day schedule overrun	Incomplete end-to-end flow	Prioritize one complete vertical slice; defer interactive 3D, CAD export, automated payments, and broad language coverage.
Sensitive photos or payment proof are exposed	Privacy and trust impact	Private storage, server-side authorization, signed URLs, short-lived links, retention limits, and audit logging.

13. Day-4 demonstration script (approximately five minutes)

1. **Landing page (30 sec):** explain the three services and choose “New Build on Empty Plot.”
2. **Voice and brief (60 sec):** record a short request in a supported priority language; show the transcript, correct a room count, and review the English brief.
3. **2D and 3D concepts (60 sec):** show the labelled SVG schematic and generated concept render. Point out the concept-only disclaimer.
4. **Interior or landscape (45 sec):** upload a sample room/garden image and generate a concept using the same image-generation adapter.
5. **Pricing and payment (60 sec):** display the itemized quote, copy the configured UPI/bank details, and submit a transaction reference/proof. Show that payment remains pending.
6. **Admin verification and release (60 sec):** verify a controlled test transaction or use a clearly labelled demo fixture; generate/preview outputs; release them; switch back to the customer view and open the released results.
7. **Security/limits (25 sec):** show the denied-unpaid API test and explain that outputs are concept images, not CAD/construction drawings.

14. Handoff and next phase after the PoC

After the four-day demonstration, do not promote the PoC to a production service without a hardening phase. Recommended next steps:

1. Run usability and speech-quality tests with real customer voices across the target languages/dialects; quantify transcription errors for names, dimensions, room counts, and local terms.
2. Engage architectural professionals to validate the room-layout generator and determine whether a real parametric CAD/BIM engine is required.
3. Introduce payment gateway integration only if desired, with verified webhooks and reconciliation; retain manual review as a separately audited option.
4. Add durable asynchronous generation jobs, idempotent retries, quota and spend limits, model/provider evaluation, observability, and customer notifications.
5. Complete privacy notice, consent, data-retention, tax, refund, and commercial terms review.
6. Conduct threat modelling, authorization/RLS review, load tests, accessibility checks, and backup/restore tests.
7. Add versioned revisions and an explicit “architect-approved” workflow if designs will be presented as professional deliverables.

15. Reference links

- Gnani AI — Speech-to-Text API
- Gnani AI — Text-to-Speech API
- Gnani AI — Model overview
- OpenRouter — Image Generation documentation
- OpenRouter — List image generation models
- OpenRouter — Unified Image API announcement and pricing notes
- Supabase — Row Level Security
- Supabase — Storage access control
- Next.js — Route Handlers

Final architecture decision

For a four-day delivery, optimize for **a secure, polished, end-to-end concept-design workflow**, not for claiming that generative images are architecturally exact. Use structured data and deterministic rendering for labelled 2D room layouts; use OpenRouter image generation for visual concepts; use Gnani for voice interfaces; use a human-verified offline-payment state; and protect generation/release at the server boundary.